

Vision-Language-Action (VLA) Models

NECKER

30 Agosto 2026

Indice

1	Definizioni Formali e Spazi Vettoriali Multi-Modali	2
1.1	Lo Spazio Visivo (Osservazione)	2
1.2	Lo Spazio Linguistico (Istruzione)	2
1.3	Lo Spazio delle Azioni (Propriocezione e Attuazione)	2
2	Fase 1: Ingestione Visiva e Proiezione Modale	3
2.1	Patchification e Vision Transformer (ViT): Dalla Griglia di Pixel ai Vettori Semantici	3
2.1.1	A. Il Problema della Complessità Quadratica e la Soluzione della Patchification	3
2.1.2	B. Flattening e Proiezione Lineare (W_{patch})	4
2.1.3	C. Iniezione della Geometria: Il Positional Embedding ($\mathbf{E}_{\text{pos}}$)	5
2.1.4	D. Il Motore di Deduzione Visiva: Self-Attention e Dinamica delle Matrici Q, K, V nel ViT	6
2.2	L'Adattatore Multi-Modale (Projector)	8
2.2.1	Allineamento e Proiezione (The Projector)	8
2.2.2	Fusione Multimodale: L'Ingresso nel Large Language Model	10
3	Fase 2: L'Emissione dell'Azione Fisica	11
3.1	Paradigma 1: Elaborazione Autoregressiva e Generazione dell'Azione	11
3.1.1	1. Action Tokenization (Discretizzazione dello Spazio di Controllo):	11
3.1.2	2. Decodifica Autoregressiva della Posa:	11
3.1.3	3. De-Quantizzazione al Dominio Fisico Continuo:	11
3.1.4	4. Attuazione a Basso Livello:	12
3.2	Paradigma 2: Generazione Continua via Conditional Flow Matching e Action Chunking	13
3.2.1	1. Topologia Architetturale a Due Livelli	13
3.2.2	2. Meccanica dell'Action Chunking: Predizione di Traiettorie Parallele	14
3.2.3	5. Esecuzione a Ciclo Chiuso: Receding Horizon e Temporal Ensembling	14
3.2.4	3. Formulazione Matematica: Conditional Flow Matching (CFM)	16
3.2.5	4. Inferenza e Integrazione Numerica dell'ODE a Runtime	16
3.3	Esecuzione sul Dispositivo (Inference Time)	19
4	Natura della Rappresentazione Visiva: Staticità Sensoriale vs. Dinamica Motoria	19
4.1	La Formulazione Quasi-Markoviana del Problema	19
4.1.1	Il paradosso dell'addestramento:	19
4.2	Limiti Fisici della Percezione a Singolo Frame	19
4.3	Strategie Architetturali di Compensazione	20
4.3.1	a) Multi-Frame Sliding Window (Stacking Temporale Limitato):	20
4.3.2	b) Iniezione di Propriocezione Storica:	20
4.3.3	c) Spostamento della Dinamica nel Futuro (Action Chunking):	20

1 Definizioni Formali e Spazi Vettoriali Multi-Modali

A differenza dei Large Language Models (LLM) puri che operano esclusivamente nello spazio discreto del linguaggio testuale, i modelli **Vision-Language-Action (VLA)** per la robotica e l'Embodied AI (come le architetture alla base di OpenVLA o SmolVLA) sono motori di ragionamento geometrico che fungono da ponte tra la percezione visiva continua, la semantica discreta e l'attuazione fisica nel mondo reale.

Per formalizzare l'architettura, dobbiamo definire i tre domini vettoriali in cui il modello opera.

1.1 Lo Spazio Visivo (Osservazione)

L'input primario è una matrice tridimensionale di pixel, catturata dai sensori (es. telecamera RGB montata sull'end-effector o nello spazio di lavoro):

$$\mathcal{I} \in \mathbb{R}^{H \times W \times C}$$

dove H e W sono la risoluzione spaziale (**height** e **width**) e $C = 3$ sono i canali colore. Nel caso di manipolazione avanzata (es. su un Franka Emika o un Dobot CR5), la percezione può estendersi a nuvole di punti 3D, ma il flusso computazionale standard inizia con tensori RGB.

1.2 Lo Spazio Linguistico (Istruzione)

L'intento umano è fornito come una sequenza di token testuali di lunghezza T_{text} , estratti dal vocabolario $\mathcal{V}$:

$$\mathbf{L} = [l_1, l_2, \dots, l_{T_{\text{text}}}] \in \mathcal{V}^{T_{\text{text}}}$$

1.3 Lo Spazio delle Azioni (Propriocezione e Attuazione)

A differenza del testo, l'output non è un concetto astratto, ma un vettore di stato cinematico. Definiamo l'azione $\mathbf{a}_t$ al tempo t come un vettore continuo nello spazio dei gradi di libertà (DoF) del manipolatore:

$$\mathbf{a}_t \in \mathbb{R}^{A_{\text{dim}}}$$

Tipicamente, $A_{\text{dim}} = 7$ (6 coordinate per la posa dell'end-effector nello spazio cartesiano $\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}$ e 1 dimensione scalare continua per lo stato del gripper).

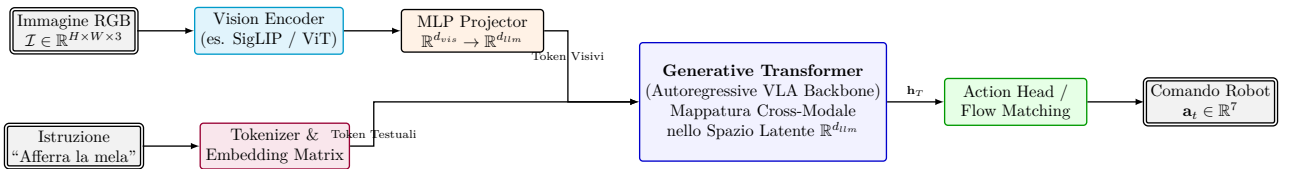


Figura 1: Macro-architettura di un modello VLA. Le modalità visiva e linguistica collassano nel medesimo spazio latente del Transformer, il quale sintetizza il contesto per produrre tensori di controllo fisico.

2 Fase 1: Ingestione Visiva e Proiezione Modale

Il modello linguistico nativo non ha il concetto di "immagine". L'immagine deve essere tradotta nella "lingua" del Transformer: una sequenza di vettori densi in $\mathbb{R}^d$.

2.1 Patchification e Vision Transformer (ViT): Dalla Griglia di Pixel ai Vettori Semantici

Il compito del **Vision Encoder** è tradurre la matrice tridimensionale continua della telecamera in una sequenza discreta di vettori.

2.1.1 A. Il Problema della Complessità Quadratica e la Soluzione della Patchification

Se tentassimo di far elaborare l'immagine pixel per pixel a un Transformer, ci scontreremmo immediatamente con il limite matematico della *Self-Attention*. Essendo la sua complessità computazionale quadratica $O(N^2)$ rispetto alla lunghezza della sequenza N , un'immagine a risoluzione standard di 224×224 pixel genererebbe una sequenza di $N = 50.176$ pixel. Il calcolo della matrice di attenzione richiederebbe oltre 2.5 miliardi di operazioni elementari per singolo layer, saturando istantaneamente la VRAM della GPU.

Per risolvere questo collo di bottiglia, l'immagine $\mathcal{I} \in \mathbb{R}^{H \times W \times C}$ viene fisicamente frammentata (sezionata) in una griglia di **patch** quadrate non sovrapposte di dimensione fissa $P \times P$ (generalmente $P = 14$ o $P = 16$).

Questa operazione, nota come *Patchification*, riduce drasticamente la lunghezza della sequenza. Il numero totale di patch estratte diventa:

$$N_p = \frac{H \times W}{P^2}$$

Nel caso di un'immagine 224×224 con patch 14×14 , otterremo esattamente $N_p = 256$ patch. L'immagine perde la sua struttura globale e diventa un array monodimensionale di "tasselli" dell'immagine originale.

ATTENZIONE: questo è utile perché nel modulo dell'attenzione non si confronta più pixel per pixel come si farebbe se si trattasse ogni pixel come singolo token, bensì si confronta patch per patch riducendo drasticamente i calcoli necessari

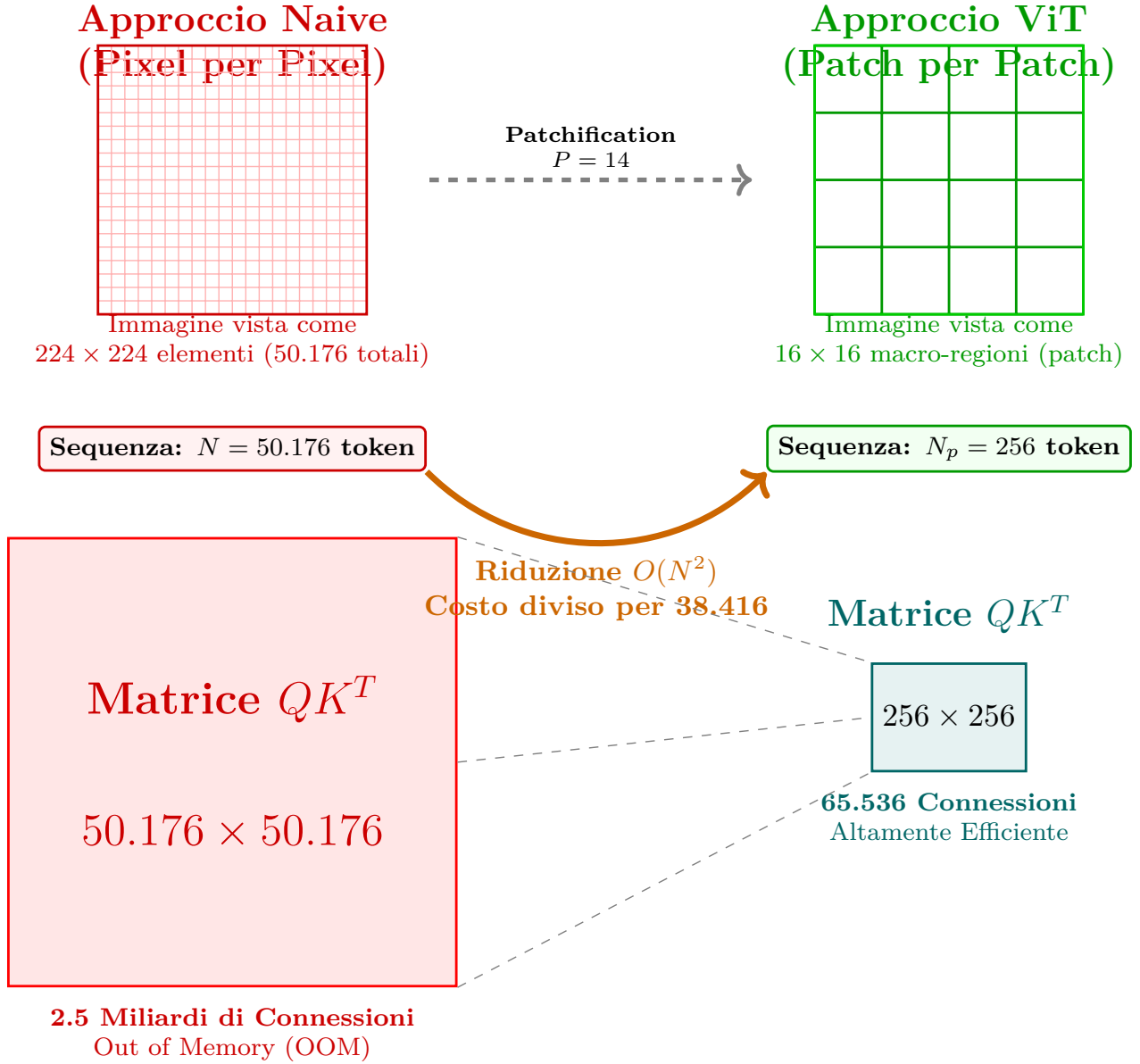


Figura 2: Rappresentazione geometrica della complessità computazionale nella *Self-Attention*. A sinistra, l'approccio denso (pixel) genera una matrice di affinità $N \times N$ fisicamente intrattabile in RAM. A destra, la *Patchification* compatta le informazioni spaziali, riducendo drasticamente le dimensioni della matrice di attenzione mantenendo intatta la capacità del modello di correlare aree distanti dell'immagine.

2.1.2 B. Flattening e Proiezione Lineare (W_{patch})

Ogni singola patch estratta è ancora un minuscolo tensore 3D di dimensioni $P \times P \times C$ (es. $14 \times 14 \times 3$). Per essere processata, deve prima subire un'operazione di **Flattening** (appiattimento in memoria), in cui le righe di pixel di ogni patch vengono srotolate e messe una in fila all'altra per formare un unico vettore riga grezzo di dimensione $P^2 \cdot C$ (es. $14 \times 14 \times 3 = 588$ numeri decimali, per esempio prima rosso poi verde poi blu).

Tuttavia, questi 588 numeri rappresentano solo valori di intensità luminosa (RGB), privi di vera semantica. Vengono quindi moltiplicati per una matrice di pesi statica W_{patch} , che agisce esattamente come la matrice di embedding W_E nei modelli linguistici:

$$\mathbf{x}_{\text{linear}}^{(i)} = \text{Flatten}(\text{Patch}_i) \cdot W_{\text{patch}} \in \mathbb{R}^{1 \times d_{\text{vis}}}$$

dove $W_{\text{patch}} \in \mathbb{R}^{(P^2 \cdot C) \times d_{\text{vis}}}$. Da un punto di vista ingegneristico, questa moltiplicazione matriciale è matematicamente equivalente a una convoluzione 2D (Conv2D) con *kernel size* pari a P e *stride* pari a P . Questa operazione proietta il tassello grezzo in uno spazio latente più ampio (tipicamente $d_{\text{vis}} = 768$ o 1152), iniziando a estrarre feature geometriche basilari (come bordi, angoli o gradienti di colore).

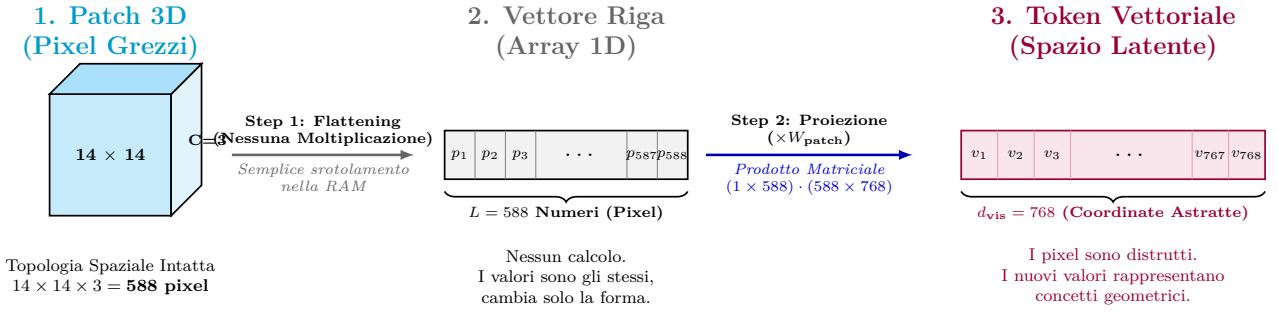


Figura 3: Scomposizione delle operazioni di pre-processing della singola patch. L'appiattimento (*Flattening*) è un'operazione di puro data-reshaping strutturale. La Proiezione (W_{patch}) è l'operazione matematica di algebra lineare che altera effettivamente la dimensionalità logica, convertendo 588 intensità visive in 768 coordinate semantiche.

2.1.3 C. Iniezione della Geometria: Il Positional Embedding ($\mathbf{E}_{\text{pos}}$)

A questo punto abbiamo 256 vettori, ma l'architettura Transformer è *permutation-invariant*: non possiede il concetto di "sopra", "sotto", "destra" o "sinistra". Se fornissimo questi vettori al modello così come sono, il robot non saprebbe distinguere se un oggetto si trova in alto a sinistra o in basso a destra, rendendo impossibile calcolare le coordinate cinematiche per afferrarlo.

Si rimedia a questa amnesia spaziale sommando algebricamente a ciascun vettore un **Positional Embedding** ($\mathbf{E}_{\text{pos}}$):

$$\mathbf{x}_p^{(i)} = \mathbf{x}_{\text{linear}}^{(i)} + \mathbf{E}_{\text{pos}}^{(i)}$$

L'embedding posizionale $\mathbf{E}_{\text{pos}}$ è una matrice di parametri appresi (o funzioni sinusoidali fisse) delle stesse dimensioni dei vettori visivi. Sommando il vettore posizionale, modifichiamo le coordinate del vettore nello spazio d_{vis} affinché esso "ricordi" permanentemente da quale zona dell'immagine originaria proviene.

Esempio Pratico di Iniezione Posizionale: Per comprendere l'effetto algebrico di $\mathbf{E}_{\text{pos}}$, consideriamo due oggetti identici (es. due cubi manipolabili) situati in regioni opposte dell'inquadratura: il primo in alto a sinistra (Patch 1) e il secondo in basso a destra (Patch 256).

A valle della sola proiezione lineare W_{patch} , i due vettori codificano esclusivamente l'aspetto visivo (colore, texture) risultando numericamente indistinguibili in un ipotetico spazio $d_{\text{vis}} = 4$:

$$\mathbf{x}_{\text{linear}}^{(1)} = [0.90, 0.10, 0.05, 0.80], \quad \mathbf{x}_{\text{linear}}^{(256)} = [0.90, 0.10, 0.05, 0.80]$$

Associando a ciascuna coordinata della griglia un vettore posizionale univoco $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{d_{\text{vis}}}$:

$$\mathbf{E}_{\text{pos}}^{(1)} = [+1.00, +1.00, 0.00, 0.00] \quad (\text{Top-Left})$$

$$\mathbf{E}_{\text{pos}}^{(256)} = [0.00, 0.00, -1.00, -1.00] \quad (\text{Bottom-Right})$$

La somma elemento per elemento genera due token visivi distinti sia nella semantica che nella collocazione spaziale:

$$\mathbf{x}_p^{(1)} = \mathbf{x}_{\text{linear}}^{(1)} + \mathbf{E}_{\text{pos}}^{(1)} = [1.90, 1.10, 0.05, 0.80]$$

$$\mathbf{x}_p^{(256)} = \mathbf{x}_{\text{linear}}^{(256)} + \mathbf{E}_{\text{pos}}^{(256)} = [0.90, 0.10, -0.95, -0.20]$$

Grazie a questa traslazione nello spazio latente, il modulo di *Self-Attention* dispone delle coordinate metriche necessarie per calcolare le traiettorie cinematiche verso il target corretto.

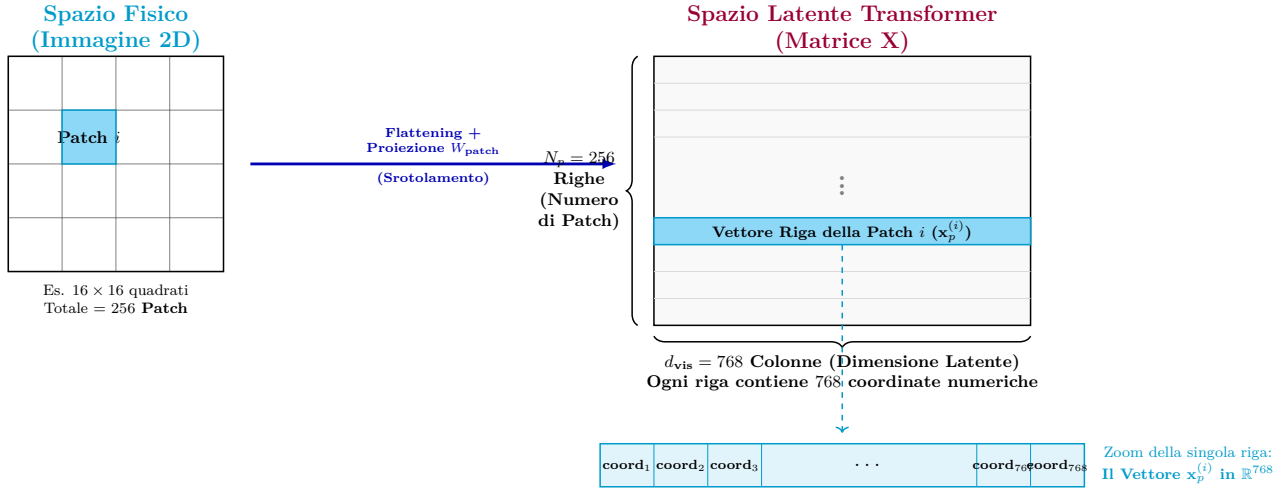


Figura 4: Relazione geometrica tra lo spazio dei pixel e lo spazio dei tensori. Le dimensioni X e Y dell'immagine originale si fondono nell'asse verticale della matrice (formando una sequenza di 256 elementi). L'asse orizzontale della matrice, invece, rappresenta la profondità semantica ($d_{\text{vis}} = 768$), ovvero il numero di coordinate matematiche utilizzate dal modello per descrivere il contenuto di quel singolo tassello.

2.1.4 D. Il Motore di Deduzione Visiva: Self-Attention e Dinamica delle Matrici Q, K, V nel ViT

I vettori $\mathbf{x}_p^{(i)} \in \mathbb{R}^{1 \times d_{\text{vis}}}$, ora dotati sia dell'impronta fotometrica iniziale che del vincolo geometrico $\mathbf{E}_{\text{pos}}$, vengono impilati nella matrice di attivazione d'ingresso $\mathbf{Z}^{(0)} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$. Questa matrice entra nella sequenza di L_{vis} blocchi di **Vision Transformer** (ViT), all'interno dei quali avviene l'astrazione semantica e relazionale.

Ogni blocco di attenzione visiva opera su tre proiezioni lineari indipendenti, parametrizzate dalle matrici di peso $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{vis}}}$:

$$Q = \mathbf{Z}^{(l-1)} W_Q, \quad K = \mathbf{Z}^{(l-1)} W_K, \quad V = \mathbf{Z}^{(l-1)} W_V$$

1. Il Ruolo di Routing di Q e K (La Geometria delle Affinità): Le matrici W_Q e W_K non trasportano le caratteristiche descrittive dell'immagine verso gli strati successivi, ma servono unicamente a determinare *chi deve parlare con chi*. Per ciascuna coppia di patch i e j , il prodotto scalare tra il vettore Query $\mathbf{q}_i$ (la richiesta di contesto della patch i) e il vettore Key $\mathbf{k}_j$ (il descrittore di compatibilità della patch j) quantifica l'affinità topologica e semantica nello spazio latente:

$$S_{i,j} = \frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} \in \mathbb{R}$$

L'applicazione della funzione Softmax lungo le righe converte questi punteggi scalari grezzi in una matrice di routing stocastica $A \in \mathbb{R}^{N_p \times N_p}$, dove ogni coefficiente $\alpha_{i,j} \in [0, 1]$ stabilisce la frazione di informazione che la patch i deve prelevare dalla patch j :

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \implies \sum_{j=1}^{N_p} \alpha_{i,j} = 1.0 \quad \forall i \in \{1, \dots, N_p\}$$

2. Il Ruolo di V (Il Vettore Valore come Contenitore di Payload Visivo): La matrice W_V assolve una funzione strutturalmente distinta: essa è l'unico canale deputato alla **trasformazione e al trasporto del contenuto percettivo reale**.

A differenza di Q e K , che collassano nella mappa di pesi scalari A , i vettori riga della matrice $V = [\mathbf{v}_1; \mathbf{v}_2; \dots; \mathbf{v}_{N_p}] \in \mathbb{R}^{N_p \times d_{\text{vis}}}$ costituiscono il *payload* informativo trasformato:

- La patch j viene proiettata linearmente tramite W_V in un nuovo sottospazio geometrico, isolando pattern complessi (es. continuità dei bordi, contrasto di profondità, appartenenza a una superficie rigida o a un punto di presa per l'end-effector).
- Il vettore $\mathbf{v}_j = \mathbf{z}_j W_V$ rappresenta il distillato fotometrico e geometrico che la patch j mette a disposizione dell'intera scena visiva.

3. Sintesi dell'Attenzione e Aggiornamento del Residual Stream: L'operazione fondamentale del modulo di attenzione visiva consiste nel moltiplicare la matrice dei pesi di routing A per la matrice dei payload V :

$$\text{Output}_{\text{Attn}} = A \cdot V \in \mathbb{R}^{N_p \times d_{\text{vis}}}$$

Analizzando la singola riga i -esima, il nuovo stato della patch i è la combinazione lineare convessa di tutti i vettori Value dell'immagine, pesati in base all'affinità:

$$\mathbf{z}_{\text{attn}}^{(i)} = \sum_{j=1}^{N_p} \alpha_{i,j} \mathbf{v}_j = \alpha_{i,1} \mathbf{v}_1 + \alpha_{i,2} \mathbf{v}_2 + \dots + \alpha_{i,N_p} \mathbf{v}_{N_p}$$

- Se la patch i contiene la texture metallica di una maniglia e le patch $j \in \mathcal{S}_{\text{porta}}$ contengono il pannello ligneo circostante, il meccanismo QK^T assegnerà pesi $\alpha_{i,j}$ elevati per $j \in \mathcal{S}_{\text{porta}}$.
- Nella sommatoria pesata, il nuovo vettore $\mathbf{z}_{\text{attn}}^{(i)}$ riceve e accumula porzioni consistenti dei vettori $\mathbf{v}_j$ del legno, integrando le proprietà globali dell'anta nella rappresentazione locale della maniglia.

Questo pacchetto sintetizzato attraversa la matrice di proiezione d'uscita $W_O \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{vis}}}$ e viene addizionato al flusso continuo tramite la prima connessione residua:

$$\mathbf{z}_{\text{mid}}^{(i)} = \mathbf{z}_{\text{input}}^{(i)} + \mathbf{z}_{\text{attn}}^{(i)} W_O$$

ATTENZIONE: $\mathbf{z}_{\text{attn}}$ viene sommato a $\mathbf{z}_{\text{input}}$ per far sì che l'informazione fotometrica locale originaria venga preservata, evitando che la media globale del contesto la cancelli.

4. Stabilizzazione RMSNorm e Modulo MLP (Deduzione su Singola Patch): Il vettore intermedio $\mathbf{z}_{\text{mid}}^{(i)}$ conclude la fase di comunicazione orizzontale tra le diverse aree dell'immagine, ma non costituisce ancora l'uscita del blocco. I dati devono essere elaborati dal secondo sotto-modulo: il **Feed-Forward Network (MLP)**.

Mentre la Self-Attention opera *cross-patch* (mette in relazione patch diverse), il modulo MLP lavora su ciascuna patch in **totale isolamento posizionale** (*point-wise*):

1. **Normalizzazione Intermedia:** Il vettore $\mathbf{z}_{\text{mid}}^{(i)}$ attraversa un secondo livello di normalizzazione per stabilizzare la varianza prima dell'espansione:

$$\mathbf{a}_{\text{mlp}}^{(i)} = \text{RMSNorm}(\mathbf{z}_{\text{mid}}^{(i)}) \in \mathbb{R}^{1 \times d_{\text{vis}}}$$

2. **Espansione Non-Lineare e Compressione (SwiGLU / GeLU):** Il vettore viene proiettato in una dimensione intermedia espansa d_{ff} (es. $d_{\text{ff}} \approx 4d_{\text{vis}}$) dove vengono applicate attivazioni non-lineari per elaborare pattern concettuali complessi, prima di essere ricompreso a d_{vis} :

$$\text{MLP}(\mathbf{a}_{\text{mlp}}^{(i)}) = \left(\text{Swish}(\mathbf{a}_{\text{mlp}}^{(i)} W_{\text{gate}}) \odot (\mathbf{a}_{\text{mlp}}^{(i)} W_{\text{up}}) \right) W_{\text{down}}$$

3. **Seconda Connessione Residua e Output di Layer:** Il risultato dell'MLP viene sommato algebricamente a $\mathbf{z}_{\text{mid}}^{(i)}$, chiudendo il layer l -esimo:

$$\mathbf{z}_{\text{out}}^{(i)} = \mathbf{z}_i^{(l)} = \mathbf{z}_{\text{mid}}^{(i)} + \text{MLP}(\mathbf{a}_{\text{mlp}}^{(i)})$$

5. Propagazione Parallela ai Blocchi Successivi ($l \rightarrow l+1$): Tutte le $N_p = 256$ patch completano questa sequenza ottenendo questa matrice aggiornata:

$$\mathbf{Z}^{(l)} = \begin{bmatrix} \mathbf{z}_1^{(l)} \\ \mathbf{z}_2^{(l)} \\ \vdots \\ \mathbf{z}_{N_p}^{(l)} \end{bmatrix} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$$

diventa l'input diretto $\mathbf{Z}_{\text{input}}^{(l+1)}$ per il layer successivo (ogni layer permette di estrarre features sempre più astratte). Questo ciclo si ripete in cascata per tutti gli L_{vis} blocchi del Vision Transformer fino all'estrazione della rappresentazione finale $\mathbf{V}_{\text{raw}} = \mathbf{Z}^{(L_{\text{vis}})}$.

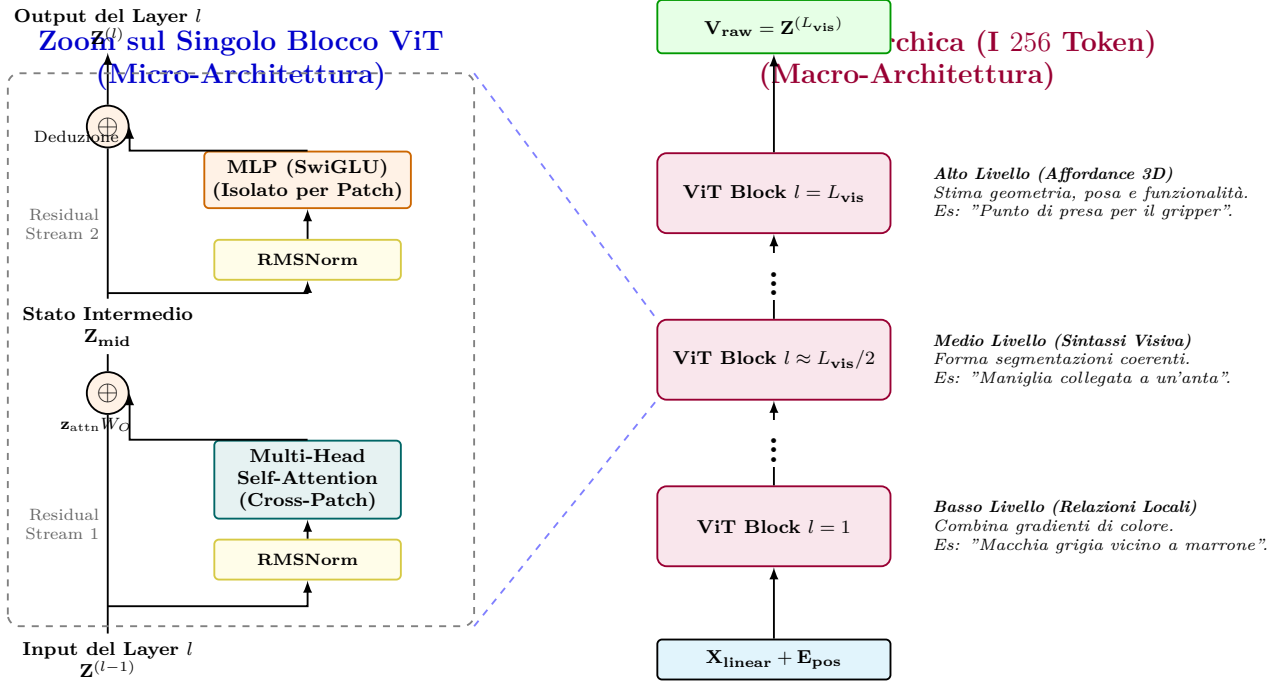


Figura 5: Sintesi del motore di ragionamento visivo. A sinistra, l'architettura interna del singolo blocco l -esimo (evidenziato il salvataggio dei dati originali tramite $\mathbf{Z}_{\text{mid}}$ e il duplice ruolo di Attention e MLP). A destra, la reiterazione ciclica di questi blocchi su tutta la profondità della rete (L_{vis}). Ad ogni passaggio (da l a $l+1$), la matematica dei layer trasforma inesorabilmente i tensori da *feature fotometriche grezze* a *strutture semantiche e cinematiche globali* pronte per la decodifica dell'azione.

2.2 L'Adattatore Multi-Modale (Projector)

Al termine dell'elaborazione del *Vision Encoder*, la matrice risultante $\mathbf{V}_{\text{raw}} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$ racchiude la semantica della scena operativa. Tuttavia, questi vettori non possono essere immessi direttamente nel blocco decisionale del modello (il Large Language Model), a causa di una profonda incompatibilità sia dimensionale che semantica:

- **Mismatch Dimensionale:** Il Vision Encoder e l'LLM possiedono spazi latenti di grandezza differente (es. $d_{\text{vis}} = 1152$ contro $d_{\text{llm}} = 4096$).
- **Mismatch Semantico:** Le coordinate di $\mathbf{V}_{\text{raw}}$ codificano l'informazione secondo la topologia visiva. L'LLM, al contrario, struttura il proprio spazio latente attorno a concetti linguistici e grammaticali.

2.2.1 Allineamento e Proiezione (The Projector)

Per colmare questo divario, la matrice visiva attraversa un modulo di adattamento chiamato **Projector** (spesso implementato come un MLP a due strati). La sua funzione matematica è proiettare i token visivi nello spazio di embedding del linguaggio.

Geometricamente, il Projector esegue una rototraslazione e un'espansione dimensionale. Dal punto di vista funzionale, esegue una vera e propria "traduzione": prende un vettore che geometricamente significa *"cilindro rosso orizzontale"* e ne altera le coordinate affinché l'LLM lo legga esattamente come se avesse letto i token di testo corrispondenti a quelle parole.

Anatomia Matematica del Multimodal Projector: Il modulo di proiezione multimodale (adottato in architetture quali LLaVA, OpenVLA e Prismatic VLMs) non si limita a un semplice riscalamento dimensionale, ma agisce come una mappa non lineare continua tra due varietà Riemanniane distinte: lo spazio delle feature visive $\mathcal{M}_{\text{vis}} \subset \mathbb{R}^{d_{\text{vis}}}$ e lo spazio dei concetti linguistici $\mathcal{M}_{\text{llm}} \subset \mathbb{R}^{d_{\text{llm}}}$.

L'architettura standard è un **Two-Layer Multilayer Perceptron (MLP)** con proiezione intermedia espansa e attivazione non lineare, formalizzata come segue:

$$\mathbf{v}_{\text{llm}}^{(i)} = f_{\text{proj}} \left(\mathbf{v}_{\text{vis}}^{(i)} \right) = \left(\sigma \left(\mathbf{v}_{\text{vis}}^{(i)} W_1 + \mathbf{b}_1 \right) \right) W_2 + \mathbf{b}_2$$

dove la cinematica dei tensori e dei parametri segue dimensioni rigorose:

- $\mathbf{v}_{\text{vis}}^{(i)} \in \mathbb{R}^{1 \times d_{\text{vis}}}$: il singolo token visivo in uscita dall'ultimo blocco del ViT (es. $d_{\text{vis}} = 1152$ per il modello visivo (vision encoder) SigLIP-SO400M).
- $W_1 \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{llm}}}$ e $\mathbf{b}_1 \in \mathbb{R}^{1 \times d_{\text{llm}}}$: la prima matrice di trasformazione affine che esegue l'allineamento di dimensione tra i due domini.
- $\sigma(\cdot) = \text{GELU}(\cdot)$: la funzione di attivazione non lineare (*Gaussian Error Linear Unit*). L'inserimento della non-linearità è critico: consente di piegare e deformare lo spazio vettoriale visivo, evitando che la corrispondenza tra feature percettive e semantica del testo sia vincolata a una rigida rototraslazione isometrica. (si usa **GELU** poiché non si ha il fenomeno dei *dying neurons* dato che la rete deve essere molto piccola e non è ridondante)
- $W_2 \in \mathbb{R}^{d_{\text{llm}} \times d_{\text{llm}}}$ e $\mathbf{b}_2 \in \mathbb{R}^{1 \times d_{\text{llm}}}$: il layer lineare finale di rifinitura nello spazio del linguaggio, che riallinea la distribuzione latente ai pesi dell'embedding dell'LLM (es. $d_{\text{llm}} = 4096$ per Llama-2-7B o Qwen-2.5-7B).

L'operazione viene applicata riga per riga in parallelo sull'intera matrice $\mathbf{V}_{\text{raw}}$, preservando il numero di token N_p :

$$\mathbf{V}_{\text{llm}} = f_{\text{proj}}(\mathbf{V}_{\text{raw}}) \in \mathbb{R}^{N_p \times d_{\text{llm}}}$$

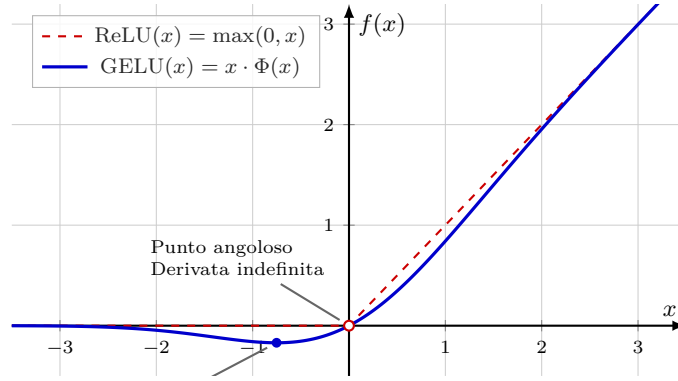


Figura 6: Confronto funzionale tra $\text{GELU}(x)$ e $\text{ReLU}(x)$. A differenza del punto di singolarità di ReLU in $x = 0$, GELU è differenziabile con continuità su tutto il dominio (C^∞) ed esibisce una regione a valori strettamente negativi per $x \in (-3, 0)$, evitando la disattivazione permanente dei gradienti (*Dying Neurons*) e garantendo stabilità di convergenza durante l'allineamento multimodale nel Projector.

Esempio Intuitivo di Traduzione Semantica: Per visualizzare l'effetto algebrico di questa transizione, consideriamo un caso giocattolo a dimensionalità ridotta ($d_{\text{vis}} = 3$, $d_{\text{llm}} = 5$) applicato a una patch che inquadra una tazza ceramica rossa manipolabile:

1. **Nello Spazio Visivo ($\mathbb{R}^3$):** Il Vision Encoder codifica esclusivamente coordinate di intensità fotometrica locale e riflessione:

$$\mathbf{v}_{\text{vis}} = [\underbrace{0.92}_{\text{Crominanza rossa}}, \underbrace{0.10}_{\text{Gradiente curvatura}}, \underbrace{0.85}_{\text{Riflesso speculare}}]$$

2. **La Trasformazione via f_{proj} :** Attraverso la moltiplicazione per W_1 , l'attivazione GELU e la matrice di riallineamento W_2 , le tre componenti fotometriche vengono combinate e ridistribuite su 5 assi latenti.
3. **Nello Spazio Linguistico ($\mathbb{R}^5$):** L'LLM struttura i propri assi vettoriali attorno a proprietà astratte e relazioni di causa-effetto:

$$\mathbf{v}_{\text{llm}} = [\underbrace{0.78}_{\text{Oggetto rigido}}, \underbrace{0.02}_{\text{Animato}}, \underbrace{0.91}_{\text{Presa/Affordance}}, \underbrace{0.63}_{\text{Contenitore}}, \underbrace{0.69}_{\text{Tonalità calda}}]$$

Il vettore risultante $\mathbf{v}_{\text{llm}}$ ha smesso di essere un descrittore ottico: ora risiede nella stessa varietà dei token testuali dell'LLM, il quale lo interpreta esattamente come se avesse letto il costrutto sintattico *"tazza rossa afferrabile"*, abilitando il successivo calcolo della traiettoria cinematica.

ATTENZIONE: questa traduzione non è una traduzione testuale, bensì si passa da vettori che indicano texture di luminosità o caratteristiche visive, a vettori che possono entrare in relazione con istruzioni vere e proprie descrivendo per esempio: Proprietà funzionali, affordance, entità manipolabili, relazioni causali

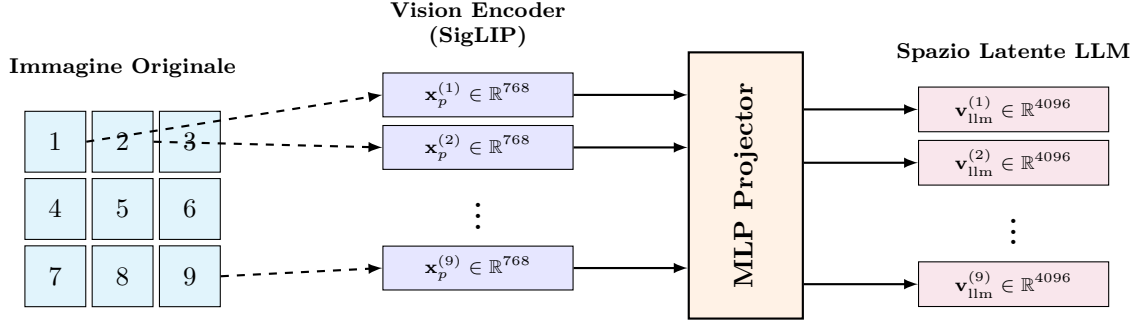


Figura 7: Meccanica della Patchification e Proiezione. L'immagine perde la sua natura bidimensionale e diviene una matrice sequenziale di vettori, compatibile con il paradigma autoregressivo.

2.2.2 Fusione Multimodale: L'Ingresso nel Large Language Model

Parallelamente all'acquisizione visiva, il robot riceve un'istruzione in linguaggio naturale dall'operatore (es. *"Afferra il blocco rosso e posizionalo a destra"*). Questa sequenza di parole attraversa il tokenizer dell'LLM e il suo *Word Embedding layer*, producendo una sequenza di token testuali:

$$\mathbf{T}_{\text{emb}} = [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_K] \in \mathbb{R}^{K \times d_{\text{llm}}}$$

dove K è il numero di subword nell'istruzione.

A questo punto, avendo uniformato la dimensionalità (d_{llm}), il modello VLA esegue la **Concatenazione Multimodale**. I vettori di embedding visivi e i vettori di embedding testuali vengono concatenati lungo l'asse della sequenza per formare un'unica macro-matrice in ingresso ai blocchi Transformer dell'LLM:

$$\mathbf{S}_{\text{input}} = [\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}] \in \mathbb{R}^{(N_p + K) \times d_{\text{llm}}}$$

Questa sequenza ibrida è il vero cuore dell'Embodied AI:

- I primi N_p vettori di embedding forniscono all'LLM il contesto visivo e geometrico dell'ambiente.
- I successivi K vettori di embedding forniscono il vincolo intenzionale (il task da eseguire).

La matrice $\mathbf{S}_{\text{input}}$ viene elaborata dalla cascata di layer causali dell'LLM, il cui meccanismo di Self-Attention permetterà all'istruzione linguistica di interrogare le *affordance* visive per prendere decisioni attuative, portando infine alla generazione autoregressiva dei **Token di Azione**.

3 Fase 2: L'Emissione dell'Azione Fisica

L'architettura classica LLM userebbe il vettore finale $\mathbf{h}_T$ per interrogare la matrice W_U (Unembedding) e produrre una nuova parola testuale. Nell'Embodied AI, dobbiamo estrarre un movimento fisico. Esistono due macro-paradigmi matematici per risolvere questa sfida.

3.1 Paradigma 1: Elaborazione Autoregressiva e Generazione dell'Azione

La macro-matrice $\mathbf{S}_{\text{input}} \in \mathbb{R}^{(N_p+K) \times d_{\text{llm}}}$ entra nella pila di L_{llm} blocchi Decoder del Large Language Model (es. Llama o Qwen).

A questo stadio, la Self-Attention opera in modo ibrido:

- I token visivi $\mathbf{V}_{\text{llm}}$ possono interagire tra loro in modo bidirezionale (o tramite maschera causale standard a seconda dell'architettura VLM).
- I token testuali $\mathbf{T}_{\text{emb}}$ interrogano i token visivi: ogni subword dell'istruzione operativa (es. *"blocco"*, *"afferra"*) calcola il prodotto scalare QK^T contro le patch visive, isolando le coordinate latenti dell'oggetto target e delle affordance di presa nello spazio di lavoro.

3.1.1 1. Action Tokenization (Discretizzazione dello Spazio di Controllo):

Un LLM standard produce predizioni su un vocabolario discreto $\mathcal{V}$ tramite una testa di classificazione lineare (*LM Head*) seguita da una Softmax. Per consentire al modello di controllare un manipolatore seriale (tipicamente a 7 gradi di libertà: $\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{gripper}$), architetture come RT-2 e OpenVLA adottano la strategia di **Action Tokenization**:

1. **Uniform Binned Discretization:** Ciascuna dimensione continua dell'azione $a_d \in [a_{\min}, a_{\max}]$ viene quantizzata uniformemente in B intervalli discreti (tipicamente $B = 256$ bin):

$$\text{bin}(a_d) = \left\lfloor \frac{a_d - a_{\min}}{a_{\max} - a_{\min}} \cdot (B - 1) \right\rfloor \in \{0, 1, \dots, B - 1\}$$

2. **Vocabolario Dedicato / Token Overwriting:** Questi 256 bin vengono mappati su 256 token riservati all'interno del vocabolario dell'LLM (spesso riciclando token testuali a frequenza d'uso nulla o aggiungendo token speciali dedicati all'estremità di $\mathcal{V}$).

3.1.2 2. Decodifica Autoregressiva della Posa:

Al termine della sequenza di input, l'LLM predice autoregressivamente i 7 token di azione consecutivi, campionando o prendendo il valore di massima verosimiglianza (argmax):

$$\mathbf{a}_{\text{discrete}} = [\tau_{\Delta x}, \tau_{\Delta y}, \tau_{\Delta z}, \tau_{\Delta \phi}, \tau_{\Delta \theta}, \tau_{\Delta \psi}, \tau_{\text{grip}}]$$

dove la probabilità di ciascun token di azione dipende condizionatamente sia dal contesto multimodale che dai sotto-comandi generati nei passi immediatamente precedenti:

$$P(\tau_k \mid \mathbf{S}_{\text{input}}, \tau_{<k}) = \text{Softmax}(\mathbf{z}_{\text{last}} W_{\text{LM}})_{\tau_k}$$

3.1.3 3. De-Quantizzazione al Dominio Fisico Continuo:

I token discreti generati vengono invertiti analiticamente per ricostruire il vettore di controllo a coordinate continue nello spazio operativo:

$$\hat{a}_d = a_{\min} + \frac{\tau_d}{B - 1} \cdot (a_{\max} - a_{\min}) \in \mathbb{R}$$

Il vettore continuo risultante:

$$\mathbf{u}_t = [\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{grip}] \in \mathbb{R}^7$$

rappresenta lo spostamento infinitesimo relativo (o la posa assoluta desiderata del TCP) nel sistema di coordinate cartesiane del robot.

3.1.4 4. Attuazione a Basso Livello:

Il vettore $\mathbf{u}_t$ viene infine inviato tramite middleware (es. ROS 2) al framework di pianificazione e controllo del manipolatore:

- La cinematica inversa computa le velocità/posizioni articolari di riferimento per i giunti: $\mathbf{q}^* = \text{IK}(\mathbf{u}_t)$.
- Gli attuatori (motori brushless / servocomandi) eseguono il movimento attraverso loop di retroazione chiusi ad alta frequenza (PD/PID su coppia e velocità), completando il ciclo percezione-decisione-azione.

Esempio Numerico End-to-End: Il Ciclo di Presa di un Cubo Per chiarire la transizione computazionale dalla semantica multimodale all'attuazione fisica, consideriamo un ciclo di controllo a frequenza discreta (10 Hz) in cui un manipolatore seriale deve eseguire un micro-avanzamento verso un cubo bersaglio sul piano di lavoro.

1. **Condizionamento Causale Multimodale:** La sequenza combinata $\mathbf{S}_{\text{input}} \in \mathbb{R}^{(256+6) \times 4096}$ attraversa i layer dell'LLM. Le teste di attenzione terminali calcolano un'alta correlazione tra il token testuale "afferra" e le patch visive indicizzate nella regione cartesiana del target. L'ultimo vettore nascosto dello strato profondo, $\mathbf{z}_{\text{last}} \in \mathbb{R}^{1 \times 4096}$, condensa l'intenzione cinematica globale.
2. **Emissione Autoregressiva dei Token di Azione:** Il modello deve emettere un'azione 7-DoF nello spazio operativo cartesiano:

$$\mathbf{u} = [\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{grip}]$$

Assumiamo per le traslazioni un range operativo limitato a $[-0.10 \text{ m}, +0.10 \text{ m}]$ discretizzato su $B = 256$ bin (da 0 a 255). L'LLM campiona in successione i token dal vocabolario riservato:

$$\begin{aligned} \tau_{\Delta x} &= 140 && \text{(lieve spostamento in avanti lungo } X) \\ \tau_{\Delta y} &= 128 && \text{(valore mediano: nessun delta laterale su } Y) \\ \tau_{\Delta z} &= 85 && \text{(abbassamento dell'end-effector verso il piano } Z) \\ \tau_{\Delta \phi}, \tau_{\Delta \theta}, \tau_{\Delta \psi} &= [128, 128, 128] && \text{(orientamento costante dell'utensile)} \\ \tau_{\text{grip}} &= 0 && \text{(stato binario: pinza aperta in fase di avvicinamento)} \end{aligned}$$

3. **De-Quantizzazione Analitica nello Spazio Continuo:** Ciascun token intero τ_d viene riconvertito in grandezza metrica reale mediante l'inversione della quantizzazione uniforme (questa formula serve per adattare il numero estratto da un intervallo fisso, in proporzione all'intervallo del giunto del robot):

$$\begin{aligned} \Delta x &= -0.10 + \frac{140}{255} \cdot (0.10 - (-0.10)) = -0.10 + \frac{140}{255} \cdot 0.20 \approx +0.0098 \text{ m} \quad (+9.8 \text{ mm}) \\ \Delta y &= -0.10 + \frac{128}{255} \cdot 0.20 \approx +0.0004 \text{ m} \approx 0.0 \text{ mm} \\ \Delta z &= -0.10 + \frac{85}{255} \cdot 0.20 \approx -0.0333 \text{ m} \quad (-33.3 \text{ mm}) \end{aligned}$$

Il vettore di spostamento infinitesimo generato per il Tool Center Point (TCP) è:

$$\mathbf{u}_t = [+0.0098 \text{ m}, 0.0 \text{ m}, -0.0333 \text{ m}, 0.0 \text{ rad}, 0.0 \text{ rad}, 0.0 \text{ rad}, 0.0]$$

4. **Cinematica Inversa e Controllo di Coppia/Velocità:** Il framework di controllo del braccio (es. nodo di traiettoria su ROS 2) riceve il delta cartesiano $\mathbf{u}_t$ a 10 Hz. Attraverso la matrice Jacobiana geometrica del braccio $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times n}$, il modulo risolve la cinematica differenziale inversa:

$$\Delta \mathbf{q} = \mathbf{J}^\dagger(\mathbf{q}) \cdot \mathbf{u}_{t(1:6)}$$

calcolando gli incrementi angolari $\Delta \mathbf{q} = [\Delta q_1, \dots, \Delta q_n]$ da inviare come setpoint ai regolatori PID dei singoli servomotori, determinando l'avvicinamento fisico del gripper all'oggetto.

ARCHITETTURA ACTION HEAD:

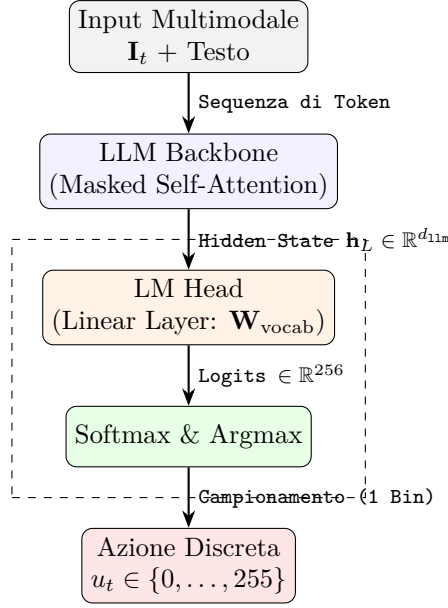


Figura 8: Architettura dell’Action Head a token discreti (Paradigma 1). La testa è un singolo strato lineare privo di memoria vettoriale.

3.2 Paradigma 2: Generazione Continua via Conditional Flow Matching e Action Chunking

I limiti intrinseci della discretizzazione (l’errore di quantizzazione su griglia fissa, la lentezza dell’inferenza autoregressiva token-per-token e la natura discontinua dei setpoint inviati ai giunti) hanno imposto una radicale riconfigurazione architetturale nei VLA di seconda generazione (come π_0 e derivati da ACT/Diffusion Policy).

In questo paradigma, il sistema non modella una distribuzione categoriale su indici discreti, ma risolve un problema di **regressione di campi vettoriali** su varietà continue, disaccoppiando il ragionamento semantico ad alta latenza dalla sintesi motoria ad alta dinamica.

3.2.1 1. Topologia Architetturale a Due Livelli

Il sistema non tenta di costringere il modello autoregressivo a generare direttamente coordinate floating-point. L’architettura viene scissa funzionalmente in due sottomoduli cooperanti:

1. **High-Level Semantic Backbone (VLM Multimodale):** È costituito dal Vision Encoder e dai blocchi Transformer pre-addestrati (es. derivati da LLaVA o PaliGemma). Riceve i frame visivi $\mathbf{V}_{\text{raw}}$ e l’istruzione $\mathbf{T}_{\text{emb}}$, computando la macro-matrice:

$$\mathbf{S}_{\text{input}} = [\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}] \in \mathbb{R}^{(N_p+K) \times d_{\text{lm}}}$$

Invece di procedere con la generazione autoregressiva tramite la classica *LM Head*, l’ultimo blocco estrae una sequenza di attivazioni latenti dense. Attraverso un’operazione di pooling attentivo o estrazione del token di fine sequenza, il backbone produce un tensore di condizionamento contestuale:

$$\mathbf{c} \in \mathbb{R}^{d_{\text{cond}}} \quad (\text{oppure una sequenza } \mathbf{C} \in \mathbb{R}^{M \times d_{\text{cond}}})$$

Questo vettore condensa l’intenzione ad alto livello (*”cosa manipolare, quali ostacoli eludere, qual è il goal finale”*). Operando su semantica astratta, questo modulo può girare a frequenze inferiori (es. 5–10 Hz).

2. **Low-Level Generative Action Head (Denoising / Vector Field Network):** È un modulo specializzato a bassa latenza (tipicamente un Transformer Decoder bidirezionale compatto da 4–8 strati o un MLP ResNet) che parametrizza il campo vettoriale:

$$\mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c})$$

La Action Head riceve in ingresso:

- Lo stato cinematico corrente $\mathbf{A}_s \in \mathbb{R}^{H \times D}$ al tempo di diffusione s .
- Lo scalare temporale continuo $s \in [0, 1]$, incorporato tramite funzioni sinusoidali di Fourier (*Sinusoidal Positional Embeddings*): $\mathbf{e}_s \in \mathbb{R}^{d_{\text{time}}}$.
- Il vettore di condizionamento $\mathbf{c}$, iniettato tramite blocchi di *Cross-Attention* o modulazione affine *AdaLN* (*Adaptive Layer Normalization*).
- Lo stato propriocettivo istantaneo del robot $\mathbf{q}_t \in \mathbb{R}^D$ (posizioni angolari attuali degli encoder o posa corrente del TCP).

3.2.2 2. Meccanica dell'Action Chunking: Predizione di Traiettorie Parallele

La differenza cardine rispetto alla decodifica causale token-per-token risiede nella generazione parallela: **gli H passi temporali futuri vengono sintetizzati simultaneamente**.

Definizione del Tensore di Traiettorie: L'output della Action Head è una matrice densa:

$$\mathbf{A} = \begin{bmatrix} \mathbf{u}_t \\ \mathbf{u}_{t+1} \\ \vdots \\ \mathbf{u}_{t+H-1} \end{bmatrix} = \begin{bmatrix} a_{t,1} & a_{t,2} & \cdots & a_{t,D} \\ a_{t+1,1} & a_{t+1,2} & \cdots & a_{t+1,D} \\ \vdots & \vdots & \ddots & \vdots \\ a_{t+H-1,1} & a_{t+H-1,2} & \cdots & a_{t+H-1,D} \end{bmatrix} \in \mathbb{R}^{H \times D}$$

dove H è l'orizzonte di pianificazione (es. $H = 50$ timesteps) e D sono i gradi di libertà attuati ($D = 7: \Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{gripper}$).

Come la rete gestisce la correlazione temporale: All'interno della Action Head, l'orizzonte temporale H non viene generato autoregressivamente in sequenza temporale (eliminando l'accumulo di errori a cascata o *compounding error*):

- La matrice rumorosa iniziale $\mathbf{A}_0 \in \mathbb{R}^{H \times D}$ viene trattata come una sequenza di H token continui.
- A ciascuna riga $h \in \{0, \dots, H-1\}$ viene sommato un embedding temporale posizionale 1D apprendibile $\mathbf{p}_h \in \mathbb{R}^{d_{\text{head}}}$, che informa la rete sulla distanza temporale relativa dal presente ($t+h$).
- I layer di **Self-Attention Bidirezionale (non causale)** permettono a ciascun timestep futuro di comunicare simultaneamente con tutti gli altri: lo step $t+10$ influenza la fluidità dello step $t+1$, imponendo continuità fisica di velocità e accelerazione lungo l'intero chunk.

3.2.3 5. Esecuzione a Ciclo Chiuso: Receding Horizon e Temporal Ensembling

Se il robot si limitasse a eseguire ciecamente l'intera matrice $\mathbf{A}^*$ per tutti gli H passi senza ri-pianificare, il controllo risulterebbe in anello aperto per un intero secondo, esponendo il manipolatore a derive sensoriali o collisioni impreviste.

Per garantire stabilità e reattività continua agli stimoli ambientali, il deployment combina due tecniche:

Receding Horizon Control (RHC): La testa generativa viene interrogata a una frequenza fissa $f_{\text{replan}} = \frac{1}{\Delta t \cdot K}$ (es. ogni $K = 8$ passi temporali, con $K \ll H$). Vengono eseguiti unicamente i primi K passi del blocco; al timestep $t+K$, il modello rilegge i frame aggiornati, estrae un nuovo vettore $\mathbf{c}$ e rigenera un nuovo orizzonte $\mathbf{A}_{t+K:t+K+H}$.

Temporal Ensembling (Fusione Esponenziale dei Piani): Poiché calcoli consecutivi generati a istanti di tempo differenti possono produrre leggere discrepanze cinematiche nei punti di giunzione, ad ogni istante t il comando effettivamente applicato ai motori $\bar{\mathbf{u}}_t$ viene calcolato come la combinazione convessa pesata di tutte le predizioni storiche sovrapposte che insistono su quell'istante:

$$\bar{\mathbf{u}}_t = \frac{\sum_{i=0}^{\min(t, H-1)} w_i \cdot \mathbf{A}_{t-i}[i]}{\sum_{i=0}^{\min(t, H-1)} w_i}$$

dove $\mathbf{A}_{t-i}[i]$ indica l'azione predetta per l'istante t all'interno del chunk calcolato i passi fa, e $w_i = \exp(-m \cdot i)$ (con $m \geq 0$) è un fattore di smorzamento esponenziale che assegna peso prioritario alle predizioni più recenti.

Questo filtraggio elimina qualsiasi salto discontinuo nei valori di derivata prima e seconda ($\dot{\mathbf{q}}, \ddot{\mathbf{q}}$), permettendo di pilotare i driver di coppia o velocità dei giunti a 50 Hz con assenza totale di oscillazioni parassite.

Esempio Numerico End-to-End: Presa Dinamica tramite Flow Matching e Action Chunking Per visualizzare concretamente il funzionamento del paradigma continuo, tracciamo l'esecuzione di un singolo ciclo di controllo per un braccio robotico a 7 DoF incaricato di afferrare un oggetto in movimento su un nastro trasportatore a 50 Hz.

1. **Estrazione del Vettore di Condizionamento ($t = 0$ s):** Il VLM multimodale elabora l'ultimo frame della telecamera e l'istruzione *"Afferra l'oggetto sul nastro"*. Attraverso la concatenazione $[\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}]$, il backbone sintetizza lo stato semantico globale in un unico vettore compatto:

$$\mathbf{c} = f_{\text{VLA}}(\mathbf{S}_{\text{input}}) \in \mathbb{R}^{4096}$$

Questo vettore non contiene coordinate numeriche esplicite, ma polarizza la Action Head verso la regione dello spazio operativo in cui intercettare il target.

2. **Inizializzazione del Chunk di Traiettoria (Puro Rumore):** Fissiamo un orizzonte di $H = 4$ passi temporali discreti a 50 Hz ($\Delta t = 0.02$ s, coprendo un intervallo futuro di 80 ms). L'azione all'istante h è data dallo scostamento cartesiano e dal gripper: $\mathbf{u}_h = [\Delta x, \Delta y, \Delta z, \Delta \phi, \Delta \theta, \Delta \psi, \text{grip}] \in \mathbb{R}^7$.

Al tempo di flusso $s = 0$, la Action Head campiona una matrice di puro rumore gaussiano standard:

$$\mathbf{A}^{(0)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \in \mathbb{R}^{4 \times 7}$$

Considerando per semplicità la sola coordinata cartesiana di avanzamento lungo X per i 4 passi futuri:

$$\mathbf{A}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} +0.82 \\ -1.15 \\ +0.34 \\ -0.61 \end{bmatrix} \quad (\text{valori casuali adimensionali})$$

3. **Integrazione dell'ODE via Metodo di Eulero ($N_{\text{steps}} = 2$ passi, $\Delta s = 0.5$):** La Action Head parametrizza il campo di velocità $\mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c})$. Supponiamo che la traiettoria fisica reale da raggiungere (in metri) sia:

$$\mathbf{A}_{[:, \Delta x]}^* = \begin{bmatrix} +0.010 \\ +0.021 \\ +0.033 \\ +0.046 \end{bmatrix} \text{ m} \quad (\text{avanzamento progressivo e continuo lungo } X)$$

- **Passo 1 ($s = 0.0$):** La rete calcola il campo vettoriale iniziale condizionato da $\mathbf{c}$ (attraverso la rete neurale dell'action head):

$$\mathbf{V}_{[:, \Delta x]}^{(0)} = \mathbf{v}_\theta(\mathbf{A}^{(0)}, 0.0, \mathbf{c}) \approx \mathbf{A}_{[:, \Delta x]}^* - \mathbf{A}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} +0.010 - 0.82 \\ +0.021 - (-1.15) \\ +0.033 - 0.34 \\ +0.046 - (-0.61) \end{bmatrix} = \begin{bmatrix} -0.810 \\ +1.171 \\ -0.307 \\ +0.656 \end{bmatrix}$$

Aggiornamento di stato con Eulero ($\Delta s = 0.5$):

$$\mathbf{A}_{[:, \Delta x]}^{(1)} = \mathbf{A}_{[:, \Delta x]}^{(0)} + 0.5 \cdot \mathbf{V}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} 0.82 + 0.5(-0.810) \\ -1.15 + 0.5(1.171) \\ 0.34 + 0.5(-0.307) \\ -0.61 + 0.5(0.656) \end{bmatrix} = \begin{bmatrix} +0.415 \\ -0.564 \\ +0.186 \\ -0.282 \end{bmatrix}$$

- **Passo 2 ($s = 0.5$):** La rete valuta la nuova velocità a metà cammino:

$$\mathbf{V}_{[:, \Delta x]}^{(1)} = \mathbf{v}_\theta(\mathbf{A}^{(1)}, 0.5, \mathbf{c}) \approx \frac{\mathbf{A}_{[:, \Delta x]}^* - \mathbf{A}_{[:, \Delta x]}^{(1)}}{1.0 - 0.5} = \begin{bmatrix} -0.810 \\ +1.170 \\ -0.306 \\ +0.656 \end{bmatrix}$$

Aggiornamento finale al target $s = 1.0$:

$$\mathbf{A}_{[:, \Delta x]}^{(2)} = \mathbf{A}_{[:, \Delta x]}^{(1)} + 0.5 \cdot \mathbf{V}_{[:, \Delta x]}^{(1)} = \begin{bmatrix} +0.010 \\ +0.021 \\ +0.033 \\ +0.046 \end{bmatrix} \text{ m}$$

In soli due passaggi di integrazione, il rumore termico è collassato sull'intera traiettoria continua multivariata $\mathbf{A}^* \in \mathbb{R}^{4 \times 7}$.

4. **Temporal Ensembling a Runtime** ($t = 0.04$ s): Dato che a ogni istante si ricalcola tutto di nuovo, all'istante $t = 2\Delta t = 0.04$ s, il controller deve emettere il comando istantaneo $\bar{\mathbf{u}}_{t=2}$.

Questo istante ricade contemporaneamente su due chunk consecutivi pianificati dal modello:

- Il chunk generato a $t = 0$, in cui l'indice relativo per $t = 2$ corrisponde a $\mathbf{u}_{t=0}[2] = +0.033$ m.
- Il nuovo chunk ricalcolato a $t = 1$ sulla base di feedback visivo aggiornato, in cui l'indice relativo per lo stesso istante è $\mathbf{u}_{t=1}[1] = +0.031$ m.

Applicando la media pesata esponenziale con coefficiente di attenuazione $m = 0.5$:

$$w_0 = e^{-0.5 \cdot 0} = 1.0 \quad (\text{predizione più recente, } t = 1), \quad w_1 = e^{-0.5 \cdot 1} \approx 0.606 \quad (\text{predizione precedente, } t = 0)$$

Il valore effettivo inviato all'interpolatore di giunti è:

$$\bar{u}_{\Delta x}(t = 2) = \frac{1.0 \cdot (+0.031) + 0.606 \cdot (+0.033)}{1.0 + 0.606} = \frac{0.031 + 0.01999}{1.606} \approx +0.0317 \text{ m} \quad (+31.7 \text{ mm})$$

La transizione tra i due piani cinematica è completamente smorzata, garantendo continuità in accelerazione e prevenendo vibrazioni meccaniche sulle giunzioni del manipolatore.

3.2.4 3. Formulazione Matematica: Conditional Flow Matching (CFM)

Il processo non si basa su salti di denoising stocastico lenti (come la classica DDPM), ma traccia una trasformazione deterministica continua tra una distribuzione base nota e la distribuzione dei dati di traiettoria ottimali.

Definizione del Path di Interpolazione: Sia $q(\mathbf{A})$ la distribuzione empirica delle dimostrazioni cinematiche esperte nel dataset e $p_0(\mathbf{A}_0) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ una distribuzione normale standard in $\mathbb{R}^{H \times D}$. Fissata una traiettoria reale $\mathbf{A}_1 \sim q(\mathbf{A})$ e un campione di rumore casuale $\mathbf{A}_0 \sim p_0(\mathbf{A}_0)$, si definisce un'interpolazione lineare controllata da una variabile temporale continua $s \in [0, 1]$ (*flow time*):

$$\mathbf{A}_s = (1 - s)\mathbf{A}_0 + s\mathbf{A}_1$$

Derivando analiticamente rispetto al tempo di flusso s , la velocità marginale della traiettoria è costante e definita da:

$$\frac{d\mathbf{A}_s}{ds} = \mathbf{A}_1 - \mathbf{A}_0$$

Il vettore $(\mathbf{A}_1 - \mathbf{A}_0) \in \mathbb{R}^{H \times D}$ rappresenta la direzione retta esatta che trasforma la configurazione di rumore $\mathbf{A}_0$ nella traiettoria cinematica corretta $\mathbf{A}_1$.

Addestramento (CFM Objective): La rete parametrica $\mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c})$ viene addestrata a regredire questo campo di velocità vettoriale minimizzando l'errore quadratico medio pesato:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}(0,1), \mathbf{A}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \mathbf{A}_1 \sim q(\mathbf{A})} \|\mathbf{v}_\theta((1 - s)\mathbf{A}_0 + s\mathbf{A}_1, s, \mathbf{c}) - (\mathbf{A}_1 - \mathbf{A}_0)\|_F^2$$

dove $\|\cdot\|_F$ denota la norma di Frobenius sul tensore $H \times D$. Si noti la straordinaria efficienza: non sono necessari cicli di diffusione iterativi durante il training; la loss valuta punti casuali s lungo la linea in un singolo forward pass.

3.2.5 4. Inferenza e Integrazione Numerica dell'ODE a Runtime

A runtime, il robot deve estrarre la traiettoria pulita senza conoscere a priori $\mathbf{A}_1$. L'estrazione equivale a integrare numericamente il problema ai valori iniziali (*IVP - Initial Value Problem*) lungo l'intervallo $s \in [0, 1]$:

$$\begin{cases} \frac{d\mathbf{A}_s}{ds} = \mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c}) \\ \mathbf{A}_{s=0} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \end{cases}$$

Poiché l'approccio Flow Matching favorisce campi vettoriali a traiettorie rettilinee (*straight flows*), l'integrazione non richiede centinaia di step, ma converge in un numero esiguo di suddivisioni $N_{\text{steps}} \in [4, 8]$ impiegando schemi classici come l'**Eulero Esplicito**:

Algorithm 1 Sintesi di Traiettoria via Integrazione ODE di Flow Matching

- 1: **Input:** Condizionamento $\mathbf{c} = f_{\text{VLA}}(\mathbf{S}_{\text{input}})$, numero di step N , passo $\Delta s = \frac{1}{N}$
 - 2: **Campionamento Iniziale:** $\mathbf{A}^{(0)} \sim \mathcal{N}(\mathbf{0}_{H \times D}, \mathbf{I}_{H \times D})$
 - 3: **for** $k = 0$ **to** $N - 1$ **do**
 - 4: $s_k \leftarrow k \cdot \Delta s$
 - 5: $\mathbf{V}^{(k)} \leftarrow \mathbf{v}_\theta(\mathbf{A}^{(k)}, s_k, \mathbf{c})$ ▷ Forward pass della Action Head
 - 6: $\mathbf{A}^{(k+1)} \leftarrow \mathbf{A}^{(k)} + \Delta s \cdot \mathbf{V}^{(k)}$ ▷ Passo di integrazione di Eulero
 - 7: **end for**
 - 8: **Output:** Matrice d'azione finale denoisificata $\mathbf{A}^* = \mathbf{A}^{(N)} \in \mathbb{R}^{H \times D}$
-

ATTENZIONE: qui si parla di derivata per spiegare come l'action head pulisce il rumore, concretamente l'action head è una rete neurale addestrata dove nell'etichetta è presente la derivata (in modo da imparare a derivare senza eseguire la formula). Usiamo come etichetta la derivata che indica la direzione che il vettore rumoroso deve prendere per pulirsi, non possiamo usare una formula statica perché a runtime non abbiamo il vettore pulito finale

ARCHITETTURA ACTION HEAD:

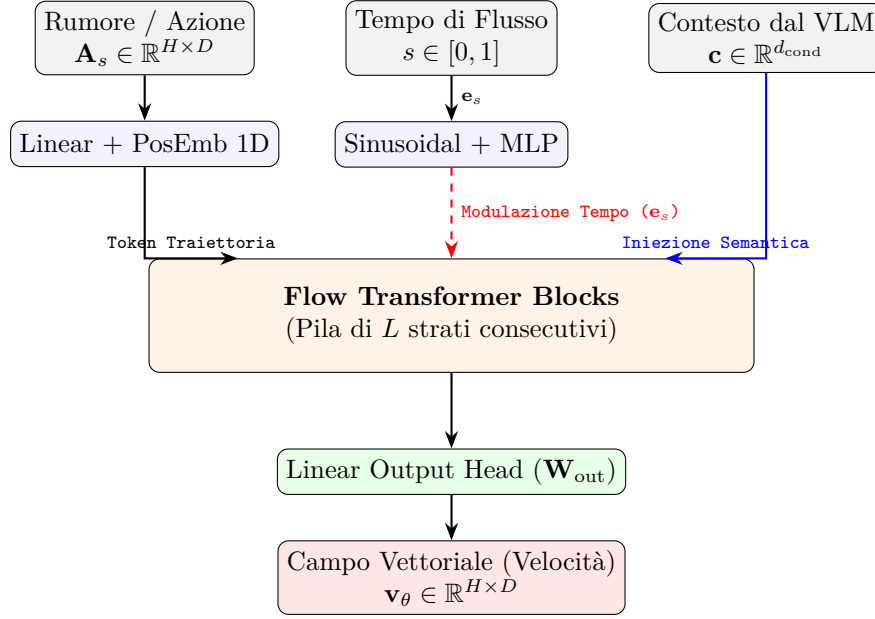


Figura 9: Macro-Architettura dell'Action Head. I tre flussi di informazione (Cinematica, Tempo, Semantica) vengono pre-processati e iniettati nel cuore della rete (il blocco Transformer).

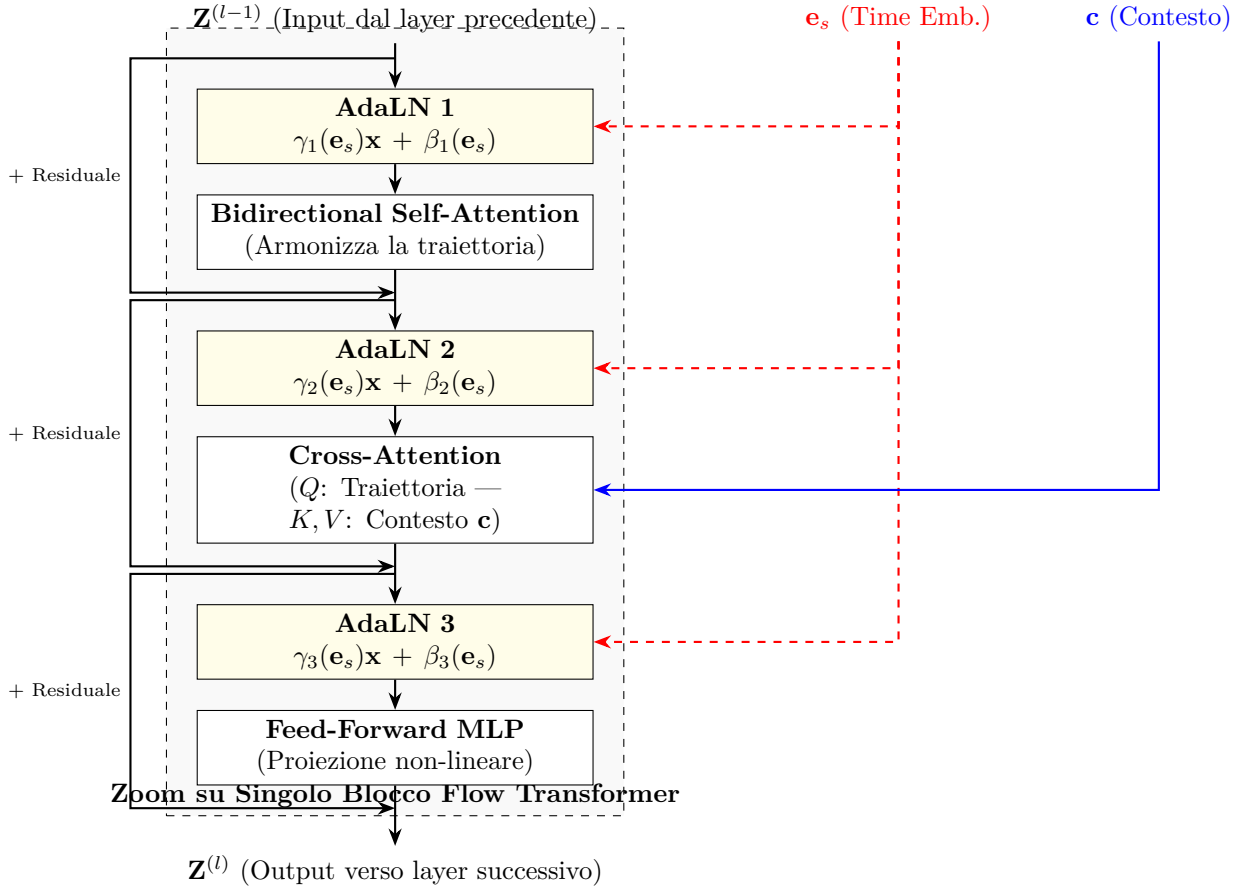


Figura 10: Dettaglio della micro-architettura di un singolo blocco Transformer. L'informazione sul tempo s scala e trasla le attivazioni neuronali tramite *AdaLN*, mentre l'informazione visiva c entra esclusivamente nel modulo di *Cross-Attention*.

3.3 Esecuzione sul Dispositivo (Inference Time)

Una volta decodificata la traiettoria continua $\mathbf{A}_{t:t+k}$, essa non viene sempre eseguita interamente ciecamente. Architetture integrate robuste sfruttano il **Temporal Ensembling**: ad ogni passo t , il VLA acquisisce una nuova immagine e ricalcola una nuova traiettoria sovrapposta. Le azioni vengono ponderate e mediate nel tempo, filtrando matematicamente le anomalie percettive e compensando la cinematica reale di bracci manipolatori come i Franka o i Dobot.

4 Natura della Rappresentazione Visiva: Staticità Sensoriale vs. Dinamica Motoria

Un aspetto critico nell'analisi dei modelli Vision-Language-Action riguarda la discrepanza tra la ricchezza dinamica del segnale nel mondo reale e la sua effettiva elaborazione computazionale: **la quasi totalità dei VLA attuali, compresa l'architettura appena studiata, opera su snapshot visivi statici (singoli frame o finestre temporali minime), rigettando l'onere computazionale di una vera elaborazione video sequenziale a lungo termine.**

4.1 La Formulazione Quasi-Markoviana del Problema

Nel paradigma canonico di imitazione (Behavior Cloning), il problema di controllo ad anello chiuso viene formulato sotto un'ipotesi strettamente o debolmente markoviana. La distribuzione delle azioni future condizionate assume la forma:

$$\pi_{\theta}(\mathbf{A}_{t:t+H} \mid \mathbf{o}_t, \mathbf{l}) \approx \pi_{\theta}(\mathbf{A}_{t:t+H} \mid \mathbf{I}_t, \mathbf{q}_t, \mathbf{l})$$

dove $\mathbf{I}_t$ è il singolo frame acquisito all'istante t , $\mathbf{q}_t$ è la proprioccezione istantanea dei giunti ed $\mathbf{l}$ è l'istruzione in linguaggio naturale.

4.1.1 Il paradosso dell'addestramento:

Durante la fase di training supervisionato su dataset di teleoperazione, le dimostrazioni registrate costituiscono a tutti gli effetti stream video continui ad alta frequenza accompagnati da telemetria temporale sincrona. Tuttavia, il processo di campionamento frammenta questa continuità:

- L'addestramento estrae tuple disaccoppiate $(\mathbf{I}_t, \mathbf{l}, \mathbf{A}_{t:t+H}^*)$ trattandole come campioni statisticamente indipendenti (*i.i.d.* batch sampling).
- Il Vision Backbone (spesso un Vision Transformer derivato da SigLIP, CLIP o DinoV2) elabora patch spaziali bidimensionali $\mathbb{R}^{C \times W \times H}$, privo di layer di attenzione temporale 3D o filtri convoluzionali lungo l'asse del tempo.
- La comprensione del task ("*cosa deve fare il manipolatore*") non scaturisce dall'interpretazione di una sequenza temporale pregressa nel video, ma dall'ancoraggio statistico tra la semantica del comando linguistico $\mathbf{l}$, la geometria spaziale statica codificata in $\mathbf{I}_t$ e l'orizzonte motorio futuro $\mathbf{A}_{t:t+H}^*$ registrato dall'operatore umano a partire da quello stato.

ATTENZIONE: in fase di addestramento quindi ogni frame contribuisce ad una previsione, si prevedono le azioni future (il chunk) e si usa la loss sulle azioni future effettivamente avvenute, si correggono i pesi in parallelo su tutto il chunk (dato che con le derivate parziali si arriva ad avere la correzione corretta in base a cosa si ha sbagliato nella singola previsione). In fase di inferenza invece, si lavora a circa 10-15 previsioni al secondo, quindi non tutti i frame vengono usati.

4.2 Limiti Fisici della Percezione a Singolo Frame

Questo disaccoppiamento impone trade-off funzionali che influenzano direttamente la stabilità del manipolatore nell'interazione con l'ambiente:

1. **Ambiguità Differenziale (Invarianza di Velocità):** Una singola immagine statica $\mathbf{I}_t$ codifica unicamente la configurazione spaziale istantanea, priva di informazioni sulla derivata prima della scena ($\dot{\mathbf{x}}_{\text{target}}$). In scenari con oggetti in moto (es. nastri trasportatori, tracking di oggetti maneggiati da un umano), un singolo fotogramma presenta una degenerazione fisica: la rete non può distinguere se il target è stazionario, in avvicinamento o in allontanamento, degradando la stima dei punti di intercettazione.

2. **Vulnerabilità alle Occlusioni Dinamiche:** Qualora il manipolatore transiti momentaneamente davanti all’obiettivo coprendo il gripper o l’oggetto bersaglio rispetto alla telecamera esterna, la politica soffre di *amnesia episodica*: non disponendo di uno stato latente ricorrente che mantenga una traccia del mondo nascosto, la predizione della traiettoria rischia il collasso per mancanza di feature geometriche valide.

4.3 Strategie Architetture di Compensazione

Per ovviare a queste limitazioni senza incorrere nei costi di inferenza proibitivi di architetture video native (come i Video Foundation Models a centinaia di frame), le implementazioni allo stato dell’arte integrano la dinamica attraverso tre meccanismi complementari:

4.3.1 a) Multi-Frame Sliding Window (Stacking Temporale Limitato):

Il Vision Encoder riceve in ingresso una sequenza corta di fotogrammi storici a intervalli regolari, ad esempio $\mathbf{S}_{\text{vis}} = [\mathbf{I}_{t-2\Delta t}, \mathbf{I}_{t-\Delta t}, \mathbf{I}_t]$. Elaborando congiuntamente questa tupla temporale minima, il meccanismo di attenzione latente estrae il flusso ottico implicito, ricostruendo i vettori di velocità degli elementi in movimento nella scena con un overhead computazionale contenuto.

4.3.2 b) Iniezione di Propriocezione Storica:

Mentre l’acquisizione e l’elaborazione dei frame visivi è vincolata a frequenze modeste (10–30 Hz), la telemetria degli encoder dei motori opera agevolmente a frequenze superiori a 500 Hz. Fornire alla rete la serie temporale recente delle posizioni e velocità dei giunti $\mathbf{Q}_{\text{hist}} = [\mathbf{q}_{t-k}, \dots, \mathbf{q}_t]$ garantisce l’osservabilità delle derivate del moto proprio del robot.

4.3.3 c) Spostamento della Dinamica nel Futuro (Action Chunking):

Poiché la percezione visiva è primariamente spaziale e a frequenza contenuta, l’onere della continuità dinamica viene trasferito interamente all’orizzonte di generazione futura. Producendo H setpoint cinematici simultanei coerenti e temporalmente legati da vincoli di non-causalità come spiegato precedentemente.